# Supplemental Source Surface

# Supplemental Source Surface I

This supplement records the top-level source surfaces a reader or reviewer should inspect before turning any prose in the main manuscript into a verifiable claim. Each surface below is authored material under version control; the generated artifacts it produces are described separately so that the boundary between hand-written source and reproducible output stays explicit.

The `src/` tree is the executable core of GNN. It is organized into 44 Python packages spanning 776 source files and roughly 231443 lines of code. Alongside these packages sit the 25 top-level step modules (0–24) that implement the numbered pipeline as thin orchestrators delegating into the packages, each module owning one stage of the progression from a parsed GNN text model through visualization, type checking, code export, and executable cognitive simulation. A claim about declared pipeline behavior should identify its source

implementation; a claim about actual execution additionally requires a source-bound command and result receipt.

The `input/` tree holds the model corpora that exercise the pipeline. Its `gnn_files/` subtree contains 12 corpus directories organized by model family, together totaling 38 example files — 29 discrete-state specifications and 9 continuous linear-Gaussian specifications, the partition the manuscript's model-kind accounting uses (sec. 3.2). The cold-start index `input/gnn_files/INDEX.md` catalogues every exemplar with its kind and task folder, and the `recursive/` directory is reserved for bounded `--autonomous` proposal-loop runs, holding no committed models. Every model file under `input/` lives in that subtree, so the 38-file count covers the whole tree. A `model_family_manifest.json` enumerates the 9 registered families and their target directories. This manifest is the

authoritative registry that downstream steps and the manuscript variable producer read when they report family counts and cross-framework coverage; it should be consulted directly rather than inferred from directory listings.

The `scripts/` tree contains the thin orchestrators that gate and reproduce the project. These include acceptance scripts that evaluate scoped pipeline runs, reliability gates that check selected semantic and coverage contracts, and the manuscript variable producer (`src/gnn/manuscript/variables.py` driven from this layer) that emits the double-brace `{{...}}` token values consumed throughout the manuscript. Treating these scripts as the source of reproduction commands keeps reported numbers bound to what the code actually computes.

## Supplemental Source Surface I

The `docs/` tree is the prose and reference surface, comprising 648 files of specification, tutorial, and design documentation for the GNN language and its Active Inference grounding (Smékal and Friedman 2023). It is the place to verify that a manuscript statement about GNN syntax or semantics matches the documented language rather than a convenient paraphrase.

The `output/` tree collects per-step pipeline artifacts: the data dumps, intermediate representations, validation reports, and the 7 figure artifacts committed under it. Generated build artifacts can be regenerated from bound inputs, while raw receipts and numerical results preserve evidence of particular runs. Reproduction requires their source and environment bindings; the directory location alone does not establish current-run evidence. Notably, `output/data/manuscript_variables.json` is where the manuscript's substituted token values are materialized.

## Authored Source Versus Generated Output I

`src/`, `scripts/`, `input/`, `docs/` and `manuscript/` are authored and reviewed. The `output/` tree holds generated artifacts and retained run evidence; `output/manuscript/` contains token-substituted copies of the authored `manuscript/` sections. Regenerate owned build artifacts when appropriate, while preserving the receipts required to support historical outcomes. The commands that regenerate the output surfaces are listed in sec. 6: the smoke run for the per-step artifacts, `scripts/z_generate_manuscript_variables.py` for the token map and the hydrated sections, `scripts.manuscript_build_figures` for the 7 manuscript figures, and the template render stage for the PDF. Which values become tokens is not a matter of taste: any quantity the producer can derive from a source surface is a token, `scripts/check_manuscript_tokens.py` fails the build when a

section hard-codes one instead, and the resulting 93-entry map is committed at `output/data/manuscript_variables.json` for audit. External references are resolved from `manuscript/references.bib`, which the same gate cross-checks against every citation key in the prose. This manuscript quotes no private or unpublished material.